AI CHAT BOT USING RAG

ROADMAP

Week 1: Data Ingestion & Chunking

Goal: Extract text from your raw files and break it into AI-digestible pieces.

Industry Best Practice: Don't just split text by a strict character count. Use "semantic chunking" (breaking at paragraph or section boundaries) so you don't cut important thoughts in half.

Member	Assigned Feature	Technical Focus
Member 1	**PDF/Document Loaders**	Use libraries like Py PDF or pdf plumber to extract raw text from files.
Member 2	**Web / API Scrapers**	Build scripts to pull data from URLs or external APIs (if needed).
Member 3	**Data Cleaner**	Write regex scripts to strip out weird characters, headers, and excessive whitespaces.
Member 4	**Chunking & Metadata**	Implement Lang Chain's text splitters. Crucially, attach "metadata" (document name, page number) to every chunk.

End of Week 1 Milestone: A unified Python script that takes a folder of PDFs and outputs a clean JSON file of text chunks with their associated metadata.

Week 2: Embeddings & Vector Search

Goal: Convert your text chunks into numbers (vectors) and build the search engine.

Industry Best Practice: Start with a lightweight, local database like Chroma DB or FAISS before jumping into complex cloud infrastructure.

Member	Assigned Feature	Technical Focus
Member 1	**Vector Database Setup**	Initialize the database, define the schema, and write the insertion logic to save the chunks.

Member	Assigned Feature	Technical Focus
Member 2	**Embedding Generation**	Connect to an embedding model (e.g., OpenAI text-embedding-3 or Hugging Face) to turn text into vectors.
Member 3	**Core Retriever Logic**	Write the function that takes a user query, embeds it, and runs a similarity search to fetch the top 3-5 results.
Member 4	**Metadata Filtering**	Enhance the search by allowing the system to filter results (e.g., "Only search documents from 2024").

End of Week 2 Milestone: A terminal-based search script. You type a question in your terminal, and it prints out the 3 most relevant paragraphs from your documents.

Week 3: Backend API & LLM Orchestration

Goal: Connect the search engine to the LLM and wrap it all in a web API.

Industry Best Practice: Strictly instruct your model in the prompt to cite its sources and to explicitly say "I don't know" if the answer isn't in the retrieved text. This prevents AI hallucinations.

Member	Assigned Feature	Technical Focus
Member 1	**API Framework**	Set up a Fast API or Flask server with endpoints for /chat and /upload.
Member 2	**Prompt Engineering**	Design the strict prompt template that forces the AI to use only the retrieved context.
Member 3	**Conversation Memory**	Build the logic to pass the last 5 chat messages to the LLM so it remembers follow-up questions.
Member 4	**LLM Integration**	Write the API call to OpenAI, Gemini, or Claude, ensuring the text streams back naturally.

End of Week 3 Milestone: A fully functioning backend API. You can send a POST request via Postman or c URL and get a fully synthesized AI answer back.

Week 4: Frontend UI & End-to-End Polish

Goal: Build the user interface, connect it to the backend, and squash bugs.

Industry Best Practice: Log the entire pipeline (from user query to retrieved chunks to final answer) so that when the UI returns a bad answer, you can see exactly where it failed.

Member	Assigned Feature	Technical Focus
Member 1	**Chat Interface**	Build the main chat window (using Streamlit, Gradio, or Next.js) for back-and-forth messaging.
Member 2	**Document Uploader UI**	Create a drag-and-drop zone so users can upload new PDFs to the system directly from the UI.
Member 3	**Source Citations**	Build a UI element that displays "Sources Used" beneath every AI answer so users can verify facts.
Member 4	**API Connections & State**	Wire the frontend buttons and inputs to the Week 3 API endpoints, handling loading states and errors.

exact tech stack you should use for this project, broken down by layer.

1. The Orchestrator & Data Pipeline

This is the framework that glues everything together—it handles reading files, chunking text, and talking to the LLM.

- **Lang Chain:** The absolute industry standard for building RAG applications. It has built-in functions for almost everything you need (document loaders, text splitters, prompt templates, and retrieval chains).
- *Alternative:* **Llama Index** (Better if your project is heavily focused on searching through highly complex documents or massive datasets).

2. The Backend API

You need a fast, reliable server to host the AI logic so the frontend can communicate with it.

- **Fast API:** This is the go-to backend framework for Python AI projects. It is incredibly fast, easy to learn, automatically generates API documentation (Swagger UI), and handles asynchronous requests perfectly (which is important when waiting for LLMs to generate text).

- **Uvicorn:** The web server that actually runs your Fast API application.

3. The Frontend (User Interface)

Since this is an AI project and not a web design project, you want a frontend framework that lets you build a chat UI purely in Python.

- **Stream lit:** The best choice for this team. You can build a ChatGPT-like interface, complete with a file uploader, sidebar, and message history, in less than 50 lines of Python code.
- *Alternative:* **Gradio** (Also great for Python AI interfaces, heavily used in the Hugging Face ecosystem).

4. Storage & Search (Vector Database)

You need a specialized database that can store vector numbers and perform semantic search.

- **Chroma (Chroma DB):** An excellent choice for group projects. It runs locally on your machine, requires zero cloud configuration, and integrates perfectly with Lang Chain.
- *Alternative:* **FAISS** (Created by Meta, purely for local fast vector search) or **Pinecone** (If you want a managed cloud database).

5. LLMs & Embeddings (The "Brain")

You need one model to turn text into vectors (embeddings) and one model to read those vectors and generate the final text (the LLM).

- **LLM (Generation):** Use the **OpenAI API (GPT-4o / GPT-4o-mini)**, **Google Gemini API**, or **Groq (Llama 3)**. Groq is highly recommended because it offers incredibly fast inference for open-source models like Llama 3, often for free or very cheap.
- **Embeddings Model: OpenAI (text-embedding-3-small)** is the easiest to use. If you want a free, local alternative that doesn't require API keys, use **Hugging Face (all-MiniLM-L6-v2 via Sentence Transformers)**.

6. Document Processors (The "Readers")

These are the small utility libraries required to pull text out of your raw files.

- **Py PDF:** A lightweight, reliable library to extract text from PDF files.
- **Beautiful Soup:** If your data pipeline requires scraping text from websites.

WORKS DISTRIBUTION:

WEEK 1:

Member 1 & 2: Focus on **Loaders**. Have one person write a script to load PDFs using Py PDF Loader, and the other write a script to load text from a website URL using Web Base Loader. Combine them into a single file so your system can read both documents and websites.

Member 3: Focus on **Cleaning**. Before the text goes into the splitter, write a quick Python function that removes junk data (like multiple blank lines, weird whitespace, or unreadable symbols) from the extracted text.

Member 4: Focus on **Chunking & Metadata**. Take the cleaned text, run it through the splitter, and ensure that every chunk remembers what file it came from (this is called "Metadata," and Lang Chain does it automatically, but you should verify it works by printing the chunks to the terminal).

WEEK 2:

Member 1 (Database Setup): Write the code that initializes Chroma DB and handles the persist_ directory so the data saves locally.

Member 2 (Embeddings): Set up the API keys and configure the Open AI Embeddings model.

Member 3 (Retriever Logic): Write the function that takes a test question, searches the database, and prints out the top 3 matches.

Member 4 (Metadata Filters): Advance the search by writing code that allows the system to filter results (e.g., "Only search chunks that came from policy.pdf").

WEEK 3:

Member 1 (API Framework): Write the Fast API structure, the Base Model data schemas, and start the local Uvicorn server.

Member 2 (Prompt Engineering): Write and tune the Chat Prompt Template to ensure the AI doesn't hallucinate facts.

Member 3 (Chain Builder): Write the Lang Chain logic that physically connects the prompt to the database retriever.

Member 4 (API Testing): Use Postman or terminal URL commands to send dummy questions to the backend and ensure it doesn't crash.

WEEK 4:

Member 1 (UI Layout): Write the st. chat message loops that draw the avatars and chat bubbles on the screen.

Member 2 (State Management): Handle st. session_ state so the conversation history is preserved correctly.

Member 3 (Backend Connection): Write the requests logic to ensure the UI successfully passes data back and forth with the Fast API server.

Member 4 (Upload Sidebar): Build a sidebar using st. sidebar .file_ uploader so users can dynamically add new PDFs from the web browser.

WEEK 5:

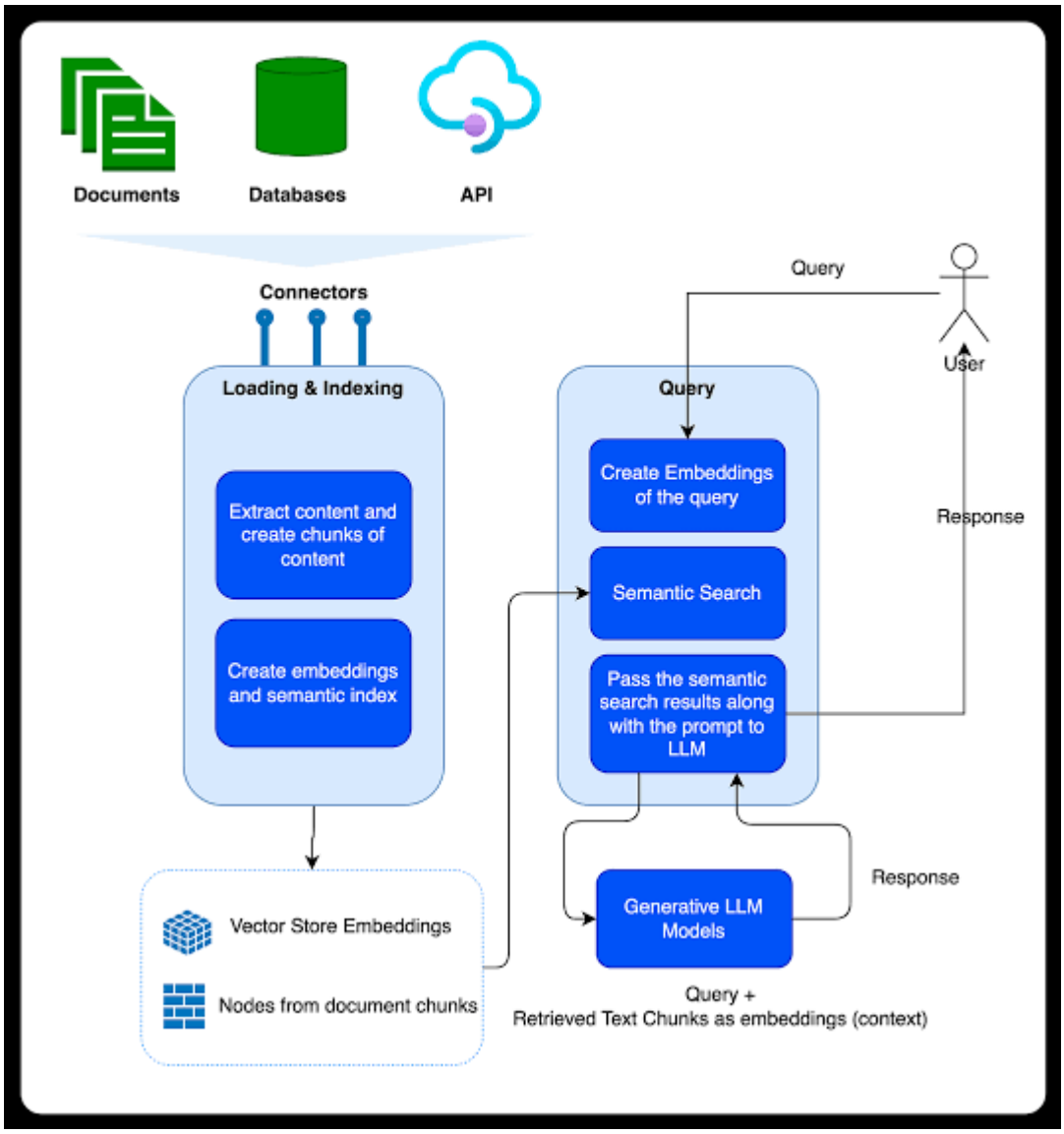
Member 1 (Adversarial Testing): Try to trick the AI. Ask it to ignore its instructions, or ask it math questions. If it hallucinates, tell Member 2.

Member 2 (Prompt Refinement): Adjust the Fast API system prompt based on Member 1's tests to make the AI bulletproof.

Member 3 (UI Polish): Make the frontend look professional. Add error popups (st. error) if the backend is offline.

Member 4 (Documentation): Write a pristine README.md file for your GitHub repository detailing exactly how a professor or recruiter can install and run your project.

The RAG Architecture Diagram:



Week 1: Setup & Ingestion Pipeline [DONE ✅]

└ Virtual Environment, Git setup, PDF parsing, Local Chroma DB

Week 2: RAG Retrieval & Prompting [IN PROGRESS ⏳]

└ Custom prompt templates, Lang Chain retrieval chains, LLM integration

Week 3: Fast API Backend & Endpoints

└ RESTful architecture, payload validation, server startup

Week 4: Stream lit UI & Deployment

└ Chat interface, chat history state, final integration & presentation prep